GramSetu AI - Technical Engineering Report

This report evaluates the technical execution, optimization architectures, memory layouts, offline strategies, and edge-processing limitations of the GramSetu AI platform.

---

1. Local AI Pipelines & Model Runtimes

GramSetu AI performs image segmentation and text extraction locally inside the browser.

SwachhAudit YOLOv8-nano Execution

- **Model Format**: ONNX (Open Neural Network Exchange).
- **File Asset**: **`public/models/yolov8n.onnx`** (14.2 MB).
- **Runtime Engine**: **`onnxruntime-web`** (configured to utilize WebGL WebGLRenderingContext backends for hardware acceleration, falling back to WebAssembly CPU execution on unsupported devices).
- **Pre-processing**: Scaled to 640x640 float32 arrays, shifting channels from interleaved RGBA to planar RGB, and normalized to the **`[0.0, 1.0]`** range.
- **Post-processing**: Custom JavaScript-based Non-Maximum Suppression (NMS) in [postProcessor.ts](file:///d:/GramSetu%20AI/app/swachh-audit/services/detection/postProcessor.ts). Discards candidates with confidence scores below **`0.25`** and resolves overlaps with an Intersection-over-Union (IoU) ratio above **`0.45`**.
- **Latency & Performance**: Currently not benchmarked on general devices. Latencies vary based on the client device GPU/CPU performance.

GramLipi Tesseract OCR Pipeline

- **Engine**: Tesseract.js (running inside a separate Web Worker thread to prevent blocking the main rendering process).
- **Asset Dependencies**: Language training set files (**`eng.traineddata`** / **`kan.traineddata`**) fetched dynamically from CDN endpoints and cached inside the browser.
- **Image Preprocessing**: Implemented in [ocrService.ts](file:///d:/GramSetu%20AI/app/gramlipi/services/ocrService.ts):
- Grayscale conversion using luminance weighting: $Y = 0.299R + 0.587G + 0.114B$.
- Contrast stretching by finding the 1st and 99th percentile luminance values in a calculated histogram, stretching the range to fill the full **`0-255`** range.
- Image upscaling to a minimum width of **`1500px`**.

JalDrishti Strip Pad extraction

- **Cropping**: Uses HTML5 Canvas context transformations in [imageProcessor.ts](file:///d:/GramSetu%20AI/app/jaldrishti/services/imageProcessor.ts) to crop the captured image down to the alignment guide overlay area.
- **Color Extraction**: Computes average RGB values for each pad by dividing the cropped canvas into target percentage regions, reading the raw pixel array, and dividing cumulative values by the pixel count.
- **Color Matching**: Matches the pad's average color against reference swatches database using Euclidean distance calculation in RGB space.

---

2. Performance & Memory Optimizations

To ensure smooth performance on low-end devices:

Memory Management

- **OCR Web Workers**: The Tesseract Web Worker is created, executed, and immediately terminated in a **`finally`** block to free up system memory:

```
try {
  // recognize text
} finally {
  await worker.terminate();
}
```

- **Image Scaling**: Preprocessing is performed on temporary HTML5 Canvas elements that are not appended to the DOM, allowing them to be garbage collected immediately after use.

Rendering & Hydration

- **Deferred State Updates**: Client-side localStorage reads are executed inside **`useEffect`** hooks and deferred using **`setTimeout`** to prevent hydration mismatches and cascading render loops in React.
- **Lazy Loading**: Code splitting is utilized for heavy external imports. For example, **`onnxruntime-web`** is imported dynamically during model loading to keep the initial page bundle size small.

---

3. Device & Browser Compatibility

The platform relies on web standards to ensure wide compatibility:

- **Browser APIs**: Utilizes standard browser APIs, including **`navigator.mediaDevices.getUserMedia`** for camera access, the **`FileReader`** API for local uploads, and the Canvas API for pixel extraction.
- **WebGL & WASM Support**: Supports modern browsers (Chrome, Safari, Firefox, Edge). Devices without GPU support fall back to the WebAssembly CPU backend.
- **Offline Strategy**: Once static assets, WebAssembly files, and model weights are cached, the core modules run without internet connection.

---

4. Technical Limitations & Future Scalability

1. **Large Model Weight Cache Limits**:

- Web browsers place limits on localStorage capacity (typically ~5MB). While local history structures are small, storing base64 images can quickly exceed this limit.
- **Mitigation**: History is saved as text results only, or images are downscaled. Future releases will migrate the SwachhAudit history logger to IndexedDB.

2. **Translation Dependency**:

- The document simplification translation engine currently interfaces with a local Ollama server. While this keeps processing local, it requires running a background desktop application.
- **Mitigation**: Future scaling plans include migrating to WebGPU-accelerated in-browser models (e.g. Transformers.js) to make translation 100% serverless.

3. **Accuracy Benchmarks**:

- Systematic accuracy measurements for YOLOv8 waste detection or color pad extraction in varied lighting conditions are currently pending. Formal accuracy benchmarks will be established in future testing iterations.